

Contents

Contents	1
Introduction	1
Architecture Overview	1
Suggested Project Structure	3
Core Workflow	4
Backend Design	6
Database Layer	8
Logging and Monitoring	8
Deployment Considerations	8
Future Technical Enhancements	9

Technical Architecture Document IPO Allotment Verification System Prepared by: Bhavya Bulani Version 1.2 · July 2026

Contents

- Introduction
- Architecture Overview
- Suggested Project Structure
- Core Workflow
- Backend Design
- Database Layer
- Logging and Monitoring
- Deployment Considerations
- Future Technical Enhancements

Introduction

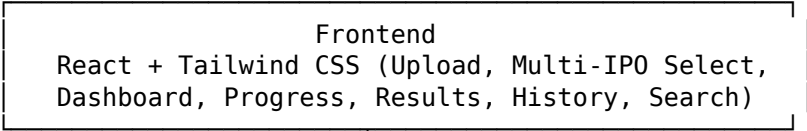
This document describes the technical architecture of the IPO Allotment Verification System: how its components fit together, how data flows through the system, and the key design decisions that make bulk allotment checking fast and resilient. It is intended for the development team building and maintaining the application.

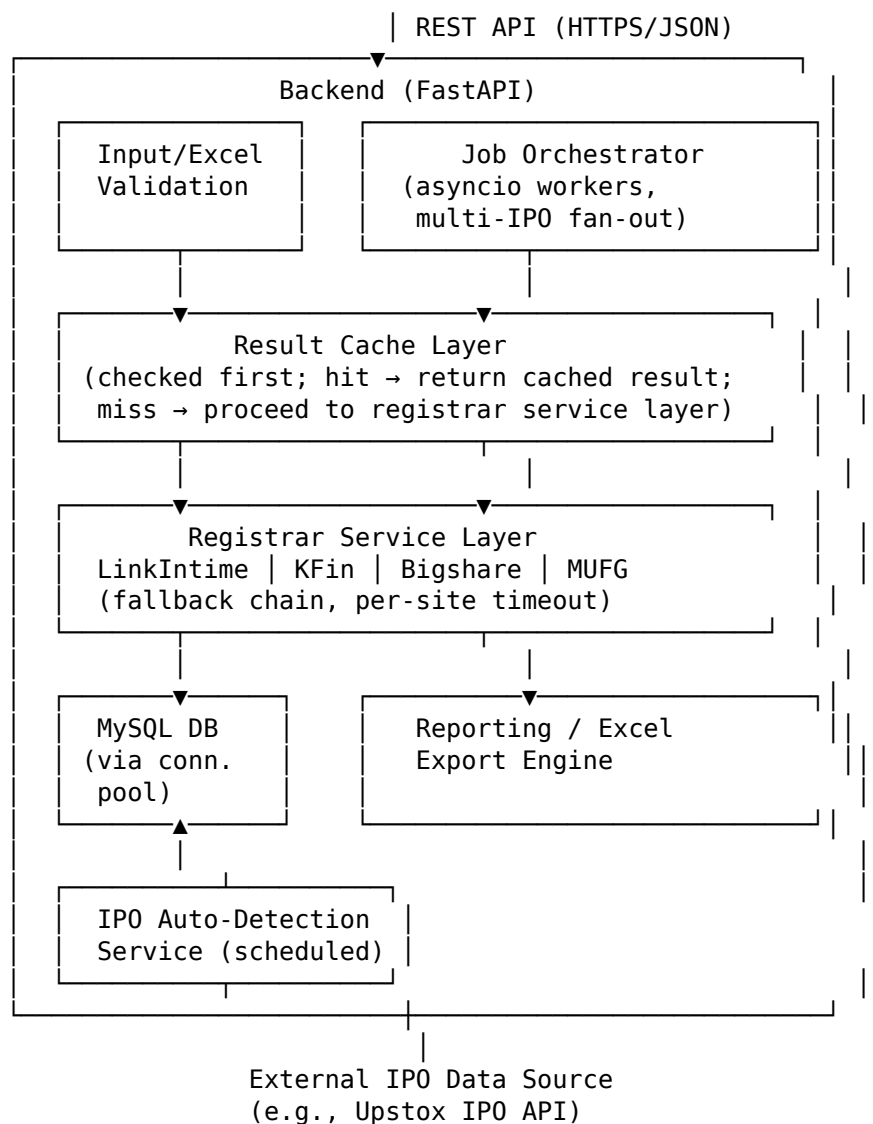
The central engineering challenge is not the user interface — it is interacting reliably with third-party registrar websites that were not designed for automated access, at a scale of hundreds or thousands of lookups per run (potentially multiplied across several IPOs at once), without freezing the application or overwhelming any single registrar.

Version 1.1 added three architectural components to the original design: an IPO Auto-Detection Service, a Multi-IPO Orchestration layer, and a Result Cache Layer. Version 1.2 changes the database layer from SQLite to **MySQL**, chosen because the system’s parallel worker pool and multi-IPO fan-out both write concurrently to the same tables — a pattern that outgrows SQLite’s single-writer model quickly, even at moderate scale.

Architecture Overview

The system follows a layered, modular architecture:





Layer Summary

Layer	Technology	Responsibility
Frontend	React + Tailwind CSS	Upload UI, single-client form, multi-IPO selector, progress display, results table, dashboard, history, search
Backend API	FastAPI (Python)	Request handling, validation orchestration, job management, report generation
Multi-IPO Orchestration	asyncio (part of Job Orchestrator)	Fanning a single client list out across all selected IPOs, tracking progress per IPO and in aggregate

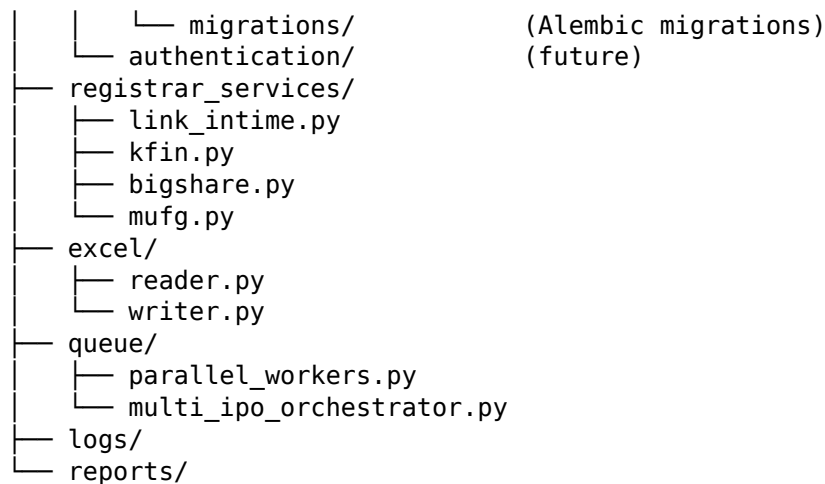
Layer	Technology	Responsibility
Result Cache Layer	In-memory cache with TTL, backed by MySQL's allotment_results cache fields	Short-circuiting duplicate registrar requests for recently-checked client/IPO pairs
IPO Auto-Detection Service	Scheduled job (backend)	Polling the external IPO data source, validating and upserting new/updated IPO records
CAPTCHA Manager	CaptchaProvider interface (Manual default; optional Solver API)	Detecting CAPTCHA challenges from a registrar and resolving them via the configured provider before the check continues
Automation Engine	Playwright (preferred) / Selenium	Browser-driven interaction with registrar sites that lack a public API
Excel Processing	pandas + openpyxl	Reading uploads, detecting columns, writing result exports
Concurrency / Queue	asyncio + worker pool	Running many client checks in parallel with bounded concurrency, across one or more IPOs
Database	MySQL (InnoDB) , accessed via a pooled connection layer	Client records, IPO master data, cached results, run history, logs
External Data Source	Upstox IPO API (or equivalent)	Synchronizing the list of current/recent IPOs, and source for auto-detection

Suggested Project Structure

```

IPO_Checker/
├── frontend/
│   ├── Upload/
│   ├── IpoMultiSelect/
│   ├── Dashboard/
│   ├── Progress/
│   └── Result/
├── backend/
│   ├── api/
│   ├── ipo_sync/
│   │   └── auto_detect.py
│   ├── cache/
│   │   └── result_cache.py
│   ├── captcha/
│   │   ├── captcha_provider.py    (interface)
│   │   ├── manual_provider.py
│   │   └── solver_provider.py     (optional, disabled by default)
│   ├── db/
│   │   ├── models.py             (SQLAlchemy ORM models, MySQL-targeted)
│   │   └── session.py            (connection pool + session factory)

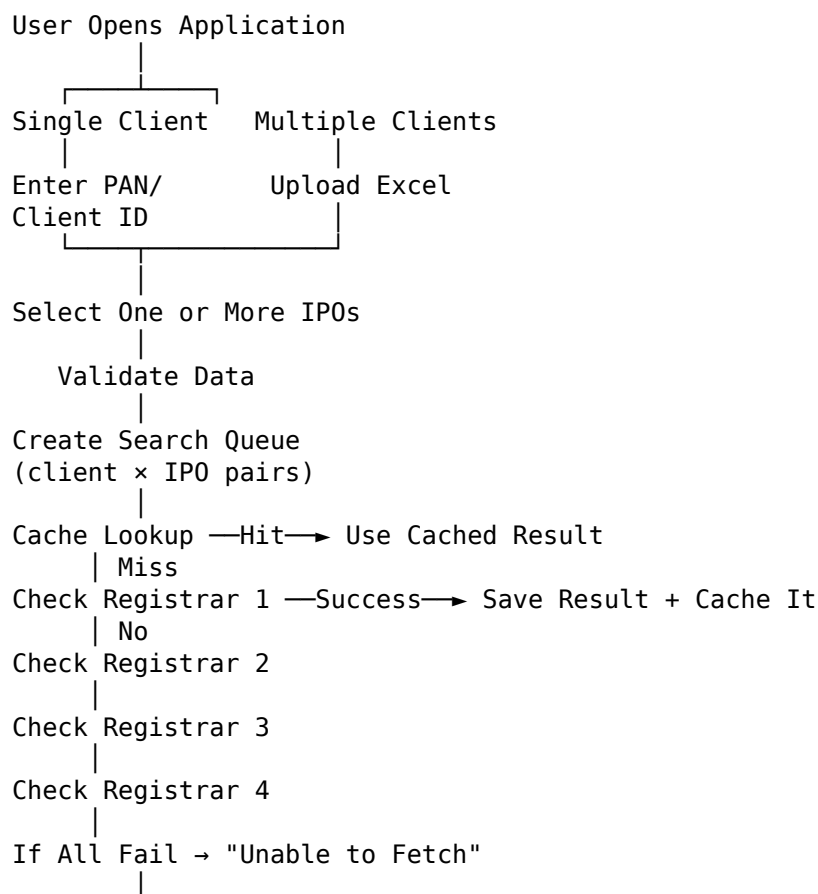
```



Each registrar is implemented as an independent module exposing a common interface (e.g., `check(pan_or_code, ipo) -> AllotmentResult`). This remains the key architectural decision that keeps the system maintainable: when a registrar changes its website, only that one module needs to change — nothing else in the pipeline is affected. The `db/` layer centralizes all MySQL access behind SQLAlchemy models and an Alembic migration history, so schema changes are versioned rather than applied ad hoc.

Core Workflow

High-Level Flow



Generate Report
|
Display Table + Download Excel

Fallback Strategy

Rather than waiting a full timeout on each registrar in turn, each attempt uses a short wait window before moving to the next registrar:

Registrar 1 → wait ~3s → Registrar 2 → wait ~3s → Registrar 3 → Registrar 4

If a client's result is found at any stage, the chain stops immediately and the result (plus which registrar supplied it) is recorded and written to the cache. If all four registrars fail, the client is recorded with an error status rather than being silently dropped; error states are not cached, so the next attempt will retry live.

Parallel Processing

For bulk runs, a configurable pool of workers (e.g., 10-30) processes many clients concurrently:

- Sequential processing of 500 clients: roughly 40 minutes.
- Parallel processing with a tuned worker pool: roughly 2-3 minutes.

Because many workers now write results to MySQL concurrently, the connection pool (see Database Layer below) must be sized to match the configured worker count, and each worker should hold its database connection for the shortest time possible (acquire → write result → release) rather than for the duration of the registrar check itself.

Multi-IPO Orchestration

When a user selects more than one IPO for a single run, the orchestrator treats the job as the Cartesian product of clients × selected IPOs:

- Each client/IPO pair is enqueued as an independent unit of work, so a failure or timeout on one pair never blocks or invalidates any other pair.
- The worker pool and per-registrar rate limits are shared across all IPOs in the run.
- Progress reporting aggregates across the whole run while remaining queryable per IPO.
- The cache lookup step is applied per client/IPO pair before any registrar is contacted, which is what keeps multi-IPO runs from linearly multiplying registrar load when clients overlap across IPOs they've already been checked against recently.

Result Caching

Before any client/IPO pair reaches the registrar service layer, the orchestrator checks the Result Cache Layer:

- Cache key: the combination of client identifier (PAN or Client Code) and IPO ID.
- Cache hit: if a non-expired cached result exists, it is returned immediately, labeled as cache-sourced, and no registrar request is made for that pair.
- Cache miss: the pair proceeds through the normal fallback chain; a successful result is written to MySQL with a fresh expiry timestamp.
- TTL (time-to-live): configurable, defaulting to a few hours.
- Error states are not cached.
- The cache is implemented as a thin layer in front of MySQL's allotment_results table (see the Database Design Document for the cache_expires_at and served_from_cache

fields); an application restart does not lose recently cached results since they live in MySQL, not in process memory alone.

IPO Auto-Detection Service

A scheduled backend job periodically polls the configured external IPO data source (e.g., the Upstox IPO API):

- New IPOs are inserted into the `ipos` table with `auto_detected = 1` and an initial unvalidated state pending a basic sanity check.
- Existing IPO records are updated if their status changes.
- Only validated IPO records are surfaced in the frontend's IPO selector.
- The job can also be triggered manually.
- Sync activity is recorded so operators can confirm the IPO list is current.

Result Categorization

Every check resolves to one of six categories (Allotted, Not Allotted, Website Error, Invalid PAN, Timeout, Server Busy), each carrying a source attribute — the specific registrar that supplied it, or “Cache” if served from the Result Cache Layer.

Backend Design

API Responsibilities

- Accept single-client requests (PAN or Client Code + selected IPO(s)).
- Accept Excel file uploads, validate file type and row count.
- Auto-detect relevant columns in the uploaded file.
- Accept one or more IPO selections per run and expand the job into client × IPO pairs.
- Check the Result Cache Layer for each pair before dispatching a registrar request.
- Expose endpoints for: run history, result search, report download, dashboard summary, and IPO list.
- Expose an endpoint (manual trigger) and internal scheduled hook for the IPO Auto-Detection Service.

Automation Engine Considerations

- Preferring an official API wherever a registrar provides one.
- Falling back to headless browser automation (Playwright) only where necessary.
- Isolating all registrar-specific logic behind a common interface.
- Applying conservative request pacing, enforced globally across the run.

CAPTCHA Handling Strategy

Some registrar sites present a CAPTCHA challenge before returning an allotment result. The architecture treats CAPTCHA handling as a pluggable concern behind a common interface, with the default configuration requiring no external dependency at all.

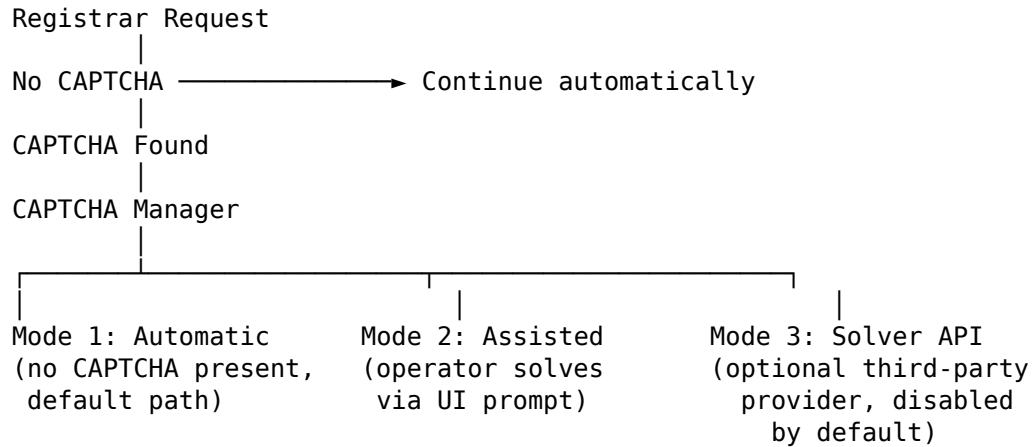
CaptchaProvider abstraction

```
class CaptchaProvider:
    def solve(self, image) -> str:
        raise NotImplementedError
```

Concrete implementations plug into this same interface:

- `ManualCaptchaProvider` — surfaces the CAPTCHA image to an operator in the UI and waits for typed input.
- `SolverProviderA` / `SolverProviderB` — optional adapters to a third-party CAPTCHA-solving API, off by default.
- Future providers can be added without any change to the registrar modules or the orchestrator.

Three operating modes



Mode	Description	Default
Automatic	Used whenever a registrar does not present a CAPTCHA; the check proceeds without any manual step.	Enabled
Assisted	When a CAPTCHA is detected, the run pauses for that client/registrar pair, the CAPTCHA image is shown to the operator in the UI, and the check resumes once solved. This mode has no external dependency and is the recommended fallback.	Enabled
Solver API (optional)	The CAPTCHA image is sent to a configured third-party solving service; the returned text is submitted automatically. Must be explicitly enabled by an administrator, off by default.	Disabled

Phased rollout

1. **Phase 1:** Support and prioritize registrars that do not require CAPTCHAs wherever possible.
2. **Phase 2:** Ship the `CaptchaManager` and the Assisted (manual-solve) mode as the standard fallback whenever a CAPTCHA is encountered.

3. **Phase 3:** If a registrar later introduces a CAPTCHA and an organization decides a third-party solver is appropriate, an administrator can enable a specific SolverProvider implementation through configuration — no application code changes required.

Database Layer

MySQL (InnoDB) is the primary database from Version 1.2 onward, replacing the SQLite-first plan used in earlier drafts. This decision was driven by the concurrency profile of the system itself: the parallel worker pool and the multi-IPO orchestrator both mean many workers can be writing `allotment_results` rows to the database at the same moment, and SQLite’s single-writer model would serialize (and eventually bottleneck) exactly the workload this system is built to speed up.

Key implications for the backend:

- All data access goes through SQLAlchemy models and Alembic migrations — no raw, engine-specific SQL scattered through the codebase — so the schema stays versioned and auditable.
- A connection pool is configured at application startup, sized to comfortably exceed the configured worker-pool concurrency (see Parallel Processing above), so workers are never blocked waiting for a free database connection.
- The `batch_ipos` join table (supporting multi-IPO runs) and the cache-related fields on `allotment_results` are expressed as native MySQL types (`ENUM`, `TINYINT(1)`, `BIGINT` for the high-volume results table) rather than SQLite’s more permissive type affinities — see the Database Design Document for the full schema.
- Because this is now the primary database rather than a placeholder, a real MySQL instance (local, containerized, or managed) must be provisioned as part of environment setup, not deferred to a later “scale-up” phase.

Logging and Monitoring

Every run maintains a structured log capturing:

- Start time and end time.
- Which registrar(s) were used per client, or whether the result was served from cache.
- Timeout count and server-busy count.
- Success count and failure count.
- Cache-hit count.
- IPO auto-detection sync events.

Deployment Considerations

- Version 1.2 is intended for internal/trusted-network deployment, since authentication and role-based access are not yet implemented.
- The automation engine (Playwright) requires a browser runtime; deployment environments must provision for this.
- A MySQL server (version 8.0+ recommended) must be provisioned alongside the application — locally via Docker for development, or a managed instance (e.g., Amazon RDS, Cloud SQL, or a college/departamental server) for anything beyond a single developer’s machine.
- Database credentials (host, user, password, database name) are supplied to the application via environment variables / a `.env` file, never hard-coded — see the Security & Access Control Document.

- Because registrar websites can change without notice, the deployment process should make it straightforward to update and redeploy an individual registrar module without a full system release.
- The IPO Auto-Detection Service’s scheduled job requires a task scheduler (e.g., APScheduler or a cron-triggered endpoint) provisioned alongside the main application.
- Database schema changes are applied via Alembic migrations as part of the deployment process, rather than manual ALTER TABLE statements against production.

Future Technical Enhancements

- A REST API layer for other internal systems to trigger checks programmatically.
- Retry with exponential backoff for transient registrar failures.
- Role-based authentication and per-user audit logging.
- Read replicas for MySQL if reporting/history queries begin to contend with write-heavy check runs.
- Distributed/shared cache (e.g., Redis) if the application is scaled across multiple instances, replacing the current database-backed cache.
- Additional CaptchaProvider implementations (third-party solver integrations) if an organization confirms this is appropriate and compliant for a given registrar.